



Список тренингов / Промпт-инжиниринг / Промпт-инжиниринг 2.0 / Модуль 7. Кейсы
(/teach/control/stream/view/id/931478147)

PE7.3 SMM-эксперт с ИИ. Часть 2

Предыдущий урок

(/pl/teach/control/lesson/view?id=338667526)

Следующий урок

(/pl/teach/control/lesson/view?id=340775305)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)

Уважаемые студенты!

В рамках данного урока могут рассматриваться примеры передачи персональных данных при регистрации на иностранных сервисах, а также обработки и хранения пользовательских данных.

В соответствии с требованиями Федерального закона № 152-ФЗ «О персональных данных», обращаем ваше внимание: в демонстрационных примерах и при выполнении домашних заданий необходимо использовать исключительно тестовые

данные или тестовые аккаунты, в которых не указаны персональные данные — используйте вымышленные ники, дополнительные данные и т.п.

Использование иностранных сервисов не является обязательным. Вы вправе выполнять домашние задания на аналогичных разрешённых платформах по вашему выбору.

Результат дня

Реализуешь SMM-ассистента в виде удобного веб-приложения.

Теория на сегодня

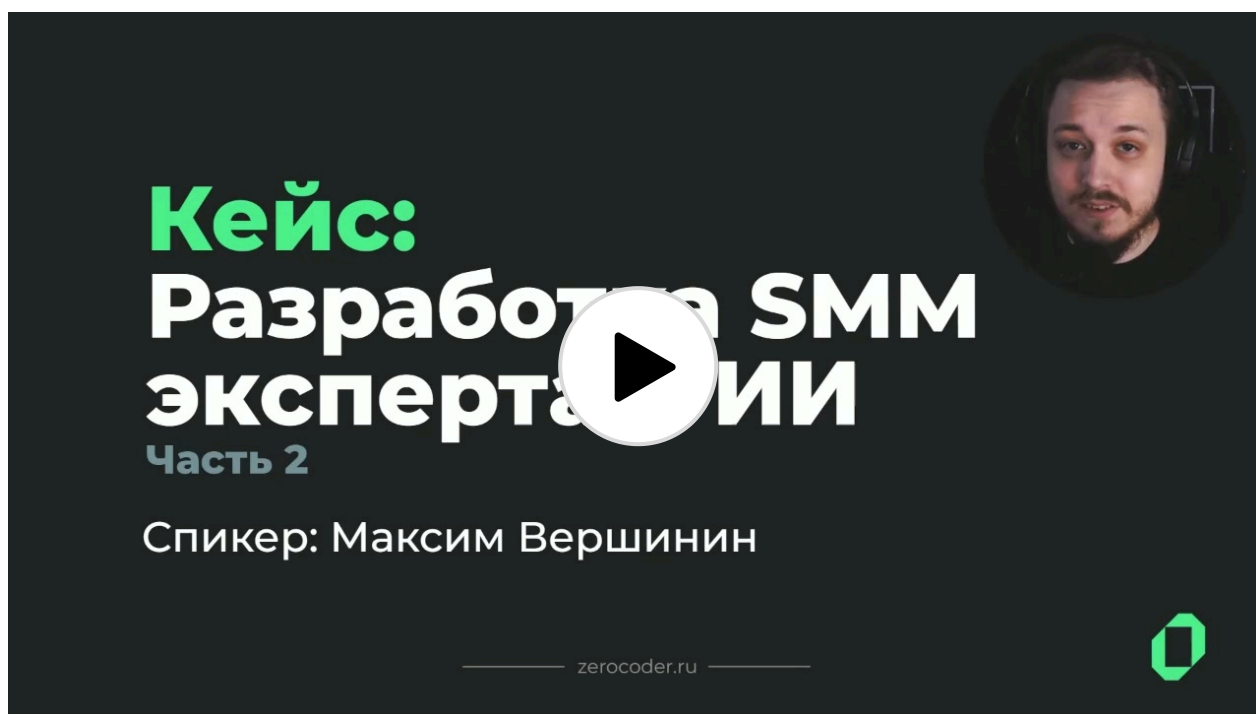
- Создание и настройка проекта с использованием Flask в PyCharm
- Создание структуры проекта с помощью ChatGPT
- Подключение функциональной части и обработка ошибок
- Создание стилизованных HTML-шаблонов с Bootstrap
- Создание аккаунта и настройка окружения на PythonAnywhere

Время изучения материалов

 Время чтения и просмотра: ~ 2 часа 15 мин

 Время выполнения задания: ~ от 1 часа

1. Вспоминаем концепцию и функционал, планируем этапы разработки



В первой части урока была разработана бизнес-логика приложения для взаимодействия с OpenAI API и социальными сетями, в частности, ВКонтакте. Сегодняшний урок посвящен созданию удобного пользовательского интерфейса для веб-приложения. Сегодняшнюю разработку можно назвать фронтенд, но на самом деле мы затронем и бэкенд приложения.

Проделанная в первой части работа уже, по сути, позволяет использовать приложение как рабочее, но пока что оно консольное. Скорее всего, конечному пользователю это будет неудобен такой формат. Поэтому сегодня мы создадим понятный интерфейс веб-приложения, доступного на любом устройстве.

Внимание, зерокодеры!

Токен от Маруси через vkhost.github.io (<http://vkhost.github.io/>) - полноценно заменяет VK Admin.

Права те же: wall, photos, groups, offline (vkhost по умолчанию запрашивает полный набор), и метод photos.getWallUploadServer доступен.

Зайдите на vkhost.github.io (<http://vkhost.github.io/>) → нажмите Маруся → разрешите доступ → скопируйте access_token из URL после редиректа → вставьте в config.py (<http://config.py/>)

План урока:

1. Вспомнить концепцию;
2. Вспомнить функционал;
3. Разбор этапов разработки веб-приложения;
4. Разработка веб-приложения.

Концепция: SMM-ассистент со встроенным ИИ, способный генерировать посты и картинки, а также работать с API социальных сетей.

Цель: Автоматизировать работу с Social Media, переложив большую часть проводимых работ на плечи ИИ.

Необходимый функционал:

- Генерация постов с установленным из заданного контекста;
- Генерация картинок для сгенерированных постов;
- Настройка автопубликации;
- Связь с метриками и аналитика.

Вспоминаем функционал, реализованный на предыдущем уроке:

- Зарегистрировались в аккаунте OpenAI и создали API-ключ.
- Создали отдельный скрипт для генерации. Генерация контента в нашем проекте теперь осуществляется через отдельный скрипт. Мы также добавили возможность генерировать описание постов, используя тот же текст и тему, на основе которых создается изображение для публикации в соцсетях.
- Настроили интеграцию с соцсетями и автопубликацию. Мы установили связь с социальными сетями и познакомились с VK-API, обнаружив, что это мощный инструмент с множеством функций. Мы можем использовать созданный API-ключ для генерации и публикации контента. Также мы настроили планирование постов для их автоматической публикации в определенное время.

- Подключили сбор статистики из социальных сетей. ****Мы научились подключаться к статистике группы ВКонтакте и извлекать различные метрики: количество просмотров, статистику аудитории по полу, возрасту, месту проживания и т.д. Эти данные можно использовать для анализа и улучшения контента.

План разработки:

- **Создать** структуру веб-приложения для пользовательского интерфейса;
- **Написать** логику работы сервера (маршруты, авторизация, хранение данных);
- **Создать** несложную frontend-часть сайта.

Создавать структуру веб-приложения мы будем на языке Python, который имеет несколько готовых фреймворков.

✅ Фреймворк — это готовый набор инструментов для разработки, который предлагает функционал, схожий с библиотеками Python. Однако фреймворк предполагает изменение структуры проекта и требует соблюдения определенных правил при разработке, обеспечивая более организованный и стандартизированный подход к созданию приложений.

Существует множество фреймворков, но для разработки на Python наиболее популярны три: Flask, Django и FastAPI. Каждый из них имеет свои особенности и требует разного уровня знаний:

- **Flask** — минималистичный и гибкий фреймворк, который подходит для небольших проектов и разработчиков, которым нужна свобода в структуре проекта.
- **Django** — самый сложный из этих фреймворков. Чтобы начать разрабатывать на Django, нужно потратить много времени на обучение взаимодействию с этим инструментом. Однако это стоит того, потому что Django — это удобный и мощный инструмент разработки, который позволяет ускорить процесс создания приложений благодаря множеству встроенных функций.
- **FastAPI** — это легковесный фреймворк, оптимизированный для создания API. Он требует глубоких знаний, так как многие функции нужно реализовывать самостоятельно. FastAPI предоставляет асинхронную

поддержку и автоматическую документацию, что делает его отличным выбором для современных веб-приложений.

Ещё один плюс фреймворка Flask в том, что он очень прост и минималистичен. Базовое приложение на Flask можно создать всего за несколько строк кода, а структура проекта остаётся чрезвычайно простой и гибкой. Если возникает необходимость в более сложной структуре, всегда можно добавить дополнительные модули Flask по мере необходимости. Например, сегодня мы будем использовать модуль Flask для авторизации, а также подключим ORM-систему Flask SQLAlchemy для работы с базами данных.

✓ ORM-система (Object-Relational Mapping) — это интерфейс, упрощающий взаимодействие с базой данных, избавляющий от необходимости писать SQL-запросы вручную. С ORM разработчику не нужно знать язык SQL, так как вся работа с базой данных осуществляется через эту систему.

Мы будем использовать и другие расширения, например, как Bcrypt, которое позволяет шифровать данные. В случае утечки данных злоумышленники не смогут воспользоваться паролями, поскольку они будут в зашифрованном виде.

Чтобы понять основы Flask, необязательно идти на специализированные курсы. В свободном доступе есть учебники по Flask и на сайте Хабр можно найти "Мега-учебник Flask", который постоянно обновляется. С его помощью можно научиться основам; также, если в уроке что-то будет непонятно, уточнить те или иные вопросы тоже можно в этом учебнике.

Дополнительные материалы и рекомендации

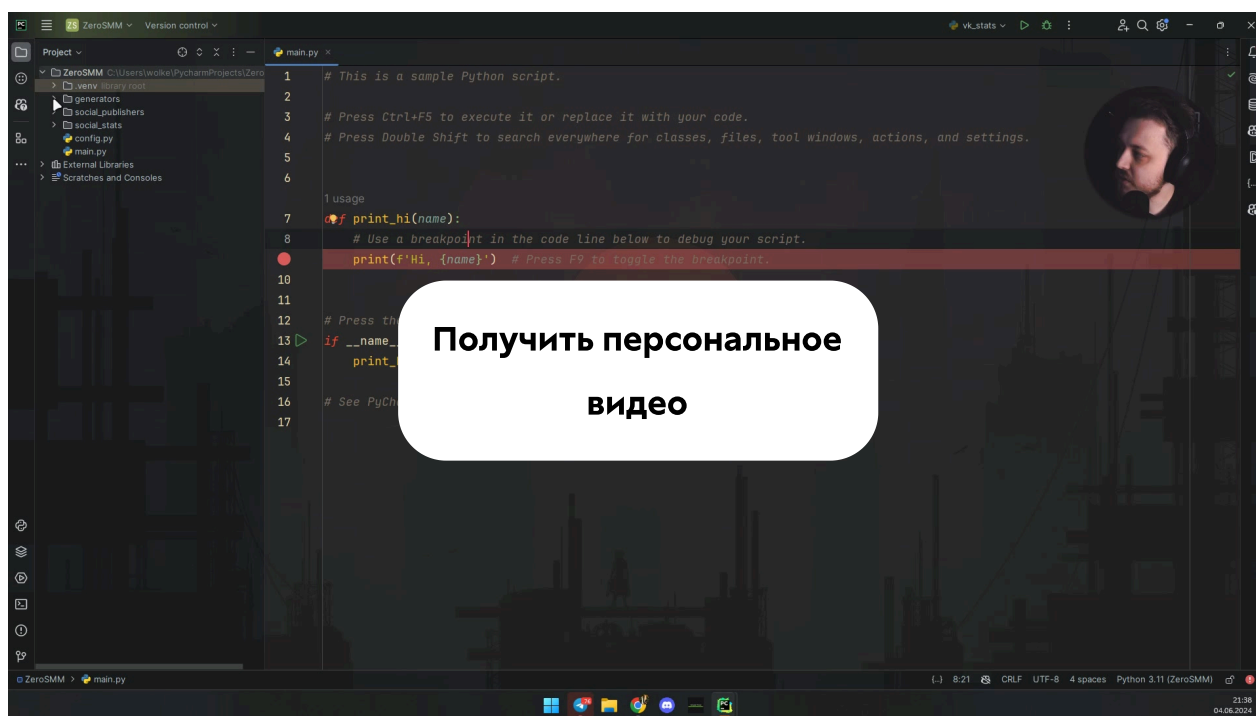
Для освоения основ Flask не обязательно записываться на специализированные курсы. Можно найти учебники по Flask в свободном доступе — например, на сайте Хабр

(<https://habr.com/ru/articles/804245/>) можно найти "Мега-учебник Flask", который постоянно обновляется. С его помощью можно овладеть основами Flask. Кроме того, если в этом уроке что-то останется непонятным, можно также обратиться к этому учебнику.

📌 **Внимание!** Далее в видео преподаватель создает код с нуля с помощью ChatGPT, то есть показывает поэтапную разработку так, как она происходит на практике. Вам не нужно писать код самостоятельно, вы можете использовать уже итоговый из архива (все файлы можно открывать любым редактором кода, но если его нет, то можно открыть просто в блокноте):

[Итоговый код для приложения SMM-эксперт.zip](https://fs19.getcourse.ru/fileservice/file/download/a/256825/sc/294/h/3ed5cb65576b64cbf8b5d4a9176eba05.zip)
(<https://fs19.getcourse.ru/fileservice/file/download/a/256825/sc/294/h/3ed5cb65576b64cbf8b5d4a9176eba05.zip>).

2. Создание и настройка проекта с использованием Flask в PyCharm



Мы находимся в PyCharm, где наш проект остался таким же, как и был. Сегодня мы заменим пустой файл `main.py` и добавим наш функционал, включая генераторы, которые генерируют изображения

и тексты, а также VK Publisher, который публикует посты в ВКонтакте и извлекает статистику. Конфигурация останется прежней, но ключи VK API будут храниться в базе данных пользователя.

Подключение Flask

Для удобства работы мы подключим Flask. Для этого нам понадобится установить новый плагин для PyCharm, который рекомендуется установить, так как он хорошо понимает структуру проекта Flask и позволяет использовать API для автодополнения кода. Для установки плагина переходим в раздел настроек (значок шестерёнки в правом верхнем углу интерфейса), выбираем вкладку **Plugins** и в разделе Marketplace в строке поиска пишем Kodeium. После установки плагина нужно перезагрузить PyCharm и войти в аккаунт Kodeium. Как только произойдёт авторизация (можно авторизоваться через Google аккаунт), плагин будет готов помочь в написании кода.

Структура проекта Flask

Самый простой способ найти структуру Flask — это просто её загуглить: "Flask project structure". Открываем, например, ссылку из документации Flask и находим здесь нужный код:


```
/home/user/Projects/flask-tutorial
├─ flaskr/
│   ├── __init__.py
│   ├── db.py
│   ├── schema.sql
│   ├── auth.py
│   ├── blog.py
│   └─ templates/
│       ├── base.html
│       ├── auth/
│       │   ├── login.html
│       │   └─ register.html
│       └─ blog/
│           ├── create.html
│           ├── index.html
│           └─ update.html
│   └─ static/
│       └─ style.css
├─ tests/
│   ├── conftest.py
│   ├── data.sql
│   ├── test_factory.py
│   ├── test_db.py
│   ├── test_auth.py
│   └─ test_blog.py
├─ .venv/
├─ pyproject.toml
└─ MANIFEST.in
```

Выглядит он внушительно, но на самом деле далеко не всё отсюда мы будем использовать. Нам понадобится создать отдельную папку, которая будет называться по имени нашего приложения (например, просто **App**), где и будет реализовываться весь функционал Flask. Коротко поговорим о том, какие файлы используются в этом коде.

Файл `__init__.py`. Папка, в которой находится этот файл, автоматически становится модулем (package) в Python. Модули можно импортировать через ключевое слово **import** и подключать

оттуда весь функционал. Это удобно при сборке комплексного приложения.

Файл `init.py` обычно оставляют пустым, но именно во Flask в этот файл помещают информацию. Здесь содержится основная информация о структуре приложения: его конфигурация, необходимые модули и т. д. В документации Flask можно найти указания о том, что именно нужно добавить в этот файл. После этого мы можем запускать модуль через главный файл, и Flask автоматически загрузит необходимые параметры, так как они уже определены в `init.py`.

Файл для работы с базами данных `db.py`. Мы не будем использовать именно такой файл. Для SQLAlchemy потребуется другой файл (`models`), в котором с помощью класса будет описана структура таблицы пользователей в базе данных.

Также из этого шаблона нам не потребуется файл `schema.sql`, потому что мы работаем с ORM-системой, которая сама создаст всё необходимое.

Файл `auth.py` из этого шаблона мы оставим для работы с авторизацией.

Файл `blog.py` представляет собой пример функционала. Так как мы разрабатываем не блог, а SMM-ассистента, здесь будет раздел, посвященный SMM.

Перейдем к созданию базовой структуры приложения Flask. Базовая структура включает:

Папки `templates` и `static` являются обязательными для проекта на Flask. В папке `static` располагаются CSS-файлы, изображения и файлы JavaScript. Мы будем использовать готовые библиотеки стилей, которые подключим позже. В папке `templates` содержатся HTML-файлы страниц веб-сайта.

Итак, заходим в PyCharm, чтобы создать такую базовую структуру, но с некоторыми модификациями.

Существует множество различных шаблонов структуры Flask, но основное требование состоит в наличии отдельного модуля `init.py`, файла для работы с моделями и базами данных, а также папок `templates` и `static`. Таковы правила оформления Flask-проекта.

Создание базовой структуры Flask в PyCharm

Находясь в PyCharm, создаём новый модуль Python Package. Назовём его `app`.

Начать проект Flask можно, просто создав новую папку (Directory), но сразу после этого в ней следует создать в ней файл `init.py`. Однако более удобным и распространённым способом является создание модуля (Python Package).

Создадим ещё несколько файлов, с которыми будем работать:

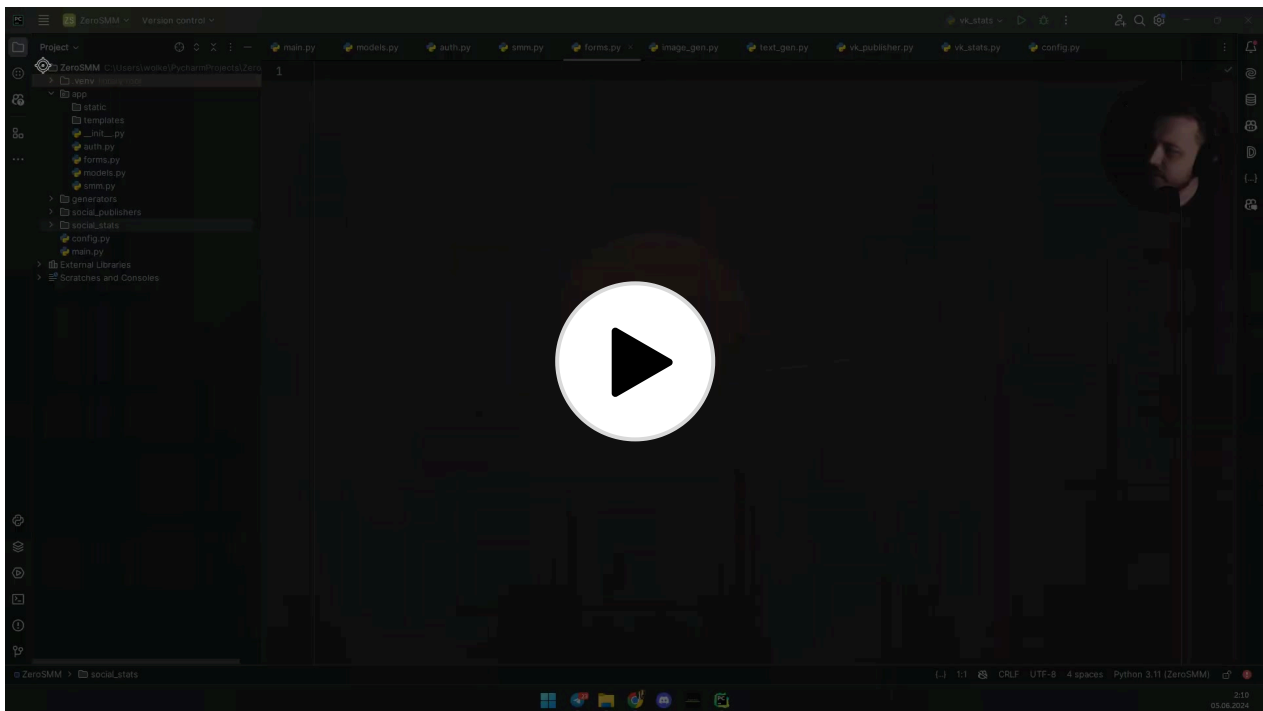
- файл `models`, в котором мы будем работать с моделями базы данных через SQLAlchemy;
- файл `auth.py` для авторизации;
- файл `smm.py`, содержащий маршруты на регистрацию и вход/выход в аккаунт;
- файл `forms.py` для форм регистрации, входа в аккаунт и указания ключей и `group id`.

Также создаём две папки:

- **templates** (она должна называться именно так, потому что именно по названию этой папки Flask находит HTML-файлы);
- **static**, внутри которой обычно создаются ещё три папки: `image (img)`, `js` и `css`. Нам эти вложенные папки создавать необязательно, поскольку папка **static** будет пустой.

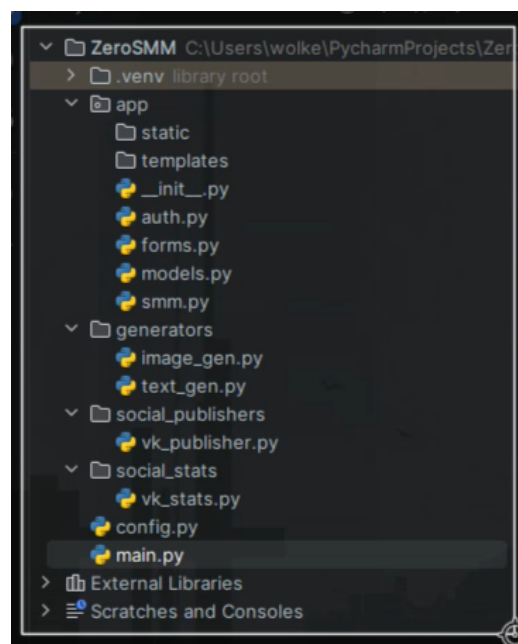
Базовая структура проекта Flask создана, теперь можно переходить к созданию кода.

3. Структура проекта. Приложение с регистрацией, авторизацией и сессиями пользователей



Делаем скриншот структуры Flask-проекта и отправляем его в ChatGPT.

Пишем запрос: "Вот такая структура проекта у меня получилась. Мне нужно помочь с реализацией скриптов flask-приложения, так как они сейчас все пустые. В нём должна быть регистрация и авторизация с возможностью после входа задать vk_api_id и vk_group_id. Нужно проработать также и страницы с



функционалом SMM-ассистента.
Не перегружай его модулями
Flask - их хочу свести к минимуму,
чтобы сделать реализацию
максимально простой".
Отправляем этот запрос.

ChatGPT предлагает использовать `flask-wtf` — это расширение для создания форм. Через него можно будет сделать удобную регистрацию и удобный вход. Также ChatGPT предлагает использовать **Blueprint**. Однако ChatGPT не предоставляет каких-либо моделей, поэтому здесь нет полноценной авторизации: чат упустил здесь базу данных.

Зададим ChatGPT вопрос: "А как пользователь сохранит сессию? Мне нужен SQLAlchemy и Bcrypt".

После уточнения ChatGPT редактирует код: помимо **flask-wtf**, он предлагает установить `flask-sqlalchemy` и `flask-bcrypt`. Однако ChatGPT, видя, что в структуре есть файл `init`, ничего в этом файле не пишет. На это нужно указать отдельно. Пишем в чат следующее сообщение: "В `init` должно создаваться приложение. Переделай всё с учётом этого".

На этот раз ChatGPT выдаёт более корректный код, который можно использовать - впрочем, для начала рассмотрим его более подробно.

Для начала откроем в PyCharm файл **init** и скопируем сюда базовый код. Однако пока что у нас нет ни одного из перечисленных в этом коде модулей. Установим их через **Python Packages**.

Заходим в **Python Packages** и устанавливаем Flask: ищем его через строку поиска и нажимаем **Install package**. Таким же образом устанавливаем **flask-wtf** и **flask-sqlalchemy** и **flask-bcrypt**.

Более подробно рассмотрим этот код. `Init` — это модуль, который запускается при импорте приложения. Как только мы воспользуемся приложением `Flask`, `init` сразу сработает. Первым делом он импортирует модули.

На следующем шаге мы видим создание двух переменных:

```
db = SQLAlchemy()
```

```
bcrypt = Bcrypt()
```

`SQLAlchemy`, как мы уже говорили, обеспечит связь с базой данных, а `Bcrypt` будет отвечать за шифрование данных.

Функция `create_app` запустится автоматически, когда мы воспользуемся приложением. Она создаст приложение `Flask` (переменная `app`).

`app.config` — здесь мы сразу применяем конфигурацию к приложению. Здесь также указывается секретный ключ, если необходим ключ доступа. В нашем случае он не требуется, поэтому мы оставим его пустым.

Также здесь в `app.config` указывается конфигурация базы данных, которая всегда остается по умолчанию как `sqlite` и `site.db`. При запуске приложения будет установлено соединение с базой данных, которую мы будем использовать.

Для инициализации базы данных прописывается `init_app` с переменной **`db`**. Таким же образом инициализируется **`bcrypt`**.

Рассмотрим следующие строки вышеприведённого кода:

```
from app.auth import auth_bp
```

```
from app.smm import smm_bp
```

Нам пока неизвестно, что это. `app` — это не только само приложение, но и `package`, в котором содержатся файлы `auth` и `smm`. К этим файлам мы будем обращаться в ходе работы. Из `app.auth` мы извлечём `auth_bp`, который ещё не определён `app.smm`, соответственно, будет извлекаться информация `smm_bp`, которой у нас тоже пока нет.

Следующие строки отвечают за регистрацию по `blueprint`.

```
app.register_blueprint(auth_bp, url_prefix='/auth')  
app.register_blueprint(smm_bp, url_prefix='/smm')
```

✓ `Blueprint` — это встроенный класс, который позволяет создавать отдельные модули по маршрутам. `Blueprint` позволяет разделить приложение на отдельные модули по функциональности.

Например, мы импортируем `auth_bp` и `smm_bp`, которые будут представлять собой `blueprint`'ы для авторизации и SMM соответственно. Затем мы регистрируем их, указывая адреса.

Когда мы регистрируем `blueprint`'ы, мы фактически указываем пути, по которым будут доступны соответствующие функции и страницы. Например, если в `blueprint` **`auth`** есть маршруты для логина и регистрации, чтобы попасть на страницу регистрации, мы должны прописать адрес вида `/auth/register`. То же самое с SMM: чтобы зайти в дашборд, нужно будет прописать адрес вида `/SMM/dashboard`.

Использование `blueprint`'ов обеспечивает модульность и независимость различных частей приложения друг от друга, что делает проект более организованным и понятным, особенно когда он становится более сложным.

Последняя строка здесь — `with app.app_context()`, куда передаётся весь контекст приложения, а затем выполняется `db.create_all()`. Это означает, что таким образом будут созданы все модели, которые

определены в файле **models** и могут потребоваться для приложения. Однако, если запустить этот код сейчас, ничего не будет создано, так как модели еще не определены.

Остальные фрагменты кода мы не будем разбирать так подробно.

Копируем из ChatGPT фрагмент кода для файла `main.py`. Вставляем этот код в файл `main` (напомним, что в проекте он стоит отдельно от приложения).

```
from app import create_app

app = create_app()

if __name__ == '__main__':
    app.run(debug=True)
```

Здесь мы импортируем функцию `create_app` из модуля **app**. Почему импортируется не `init`, а именно `app`? Потому что `init.py` (<http://init.py>) неразрывно связан с модулем `app`. Все, что определено в `init.py`, применяется ко всему модулю, поэтому мы создали пакет. Функция `create_app`, находящаяся в `init.py`, инициализирует все необходимые компоненты приложения, а `main.py` запускает это приложение.

Что такое `if __name__ == '__main__':` мы говорили в предыдущей части. Здесь мы запускаем приложение в режиме отладки (`debug mode`). В случае развертывания (`deploy`), приложение будет запускаться на определенном хостинге с указанным адресом и портом.

Копируем из ChatGPT код для `auth.py`. В этом фрагменте кода используется множество модулей, однако на данный момент у нас нет данных для работы с `User`, `db` и `bcrypt`. Это происходит потому, что модели и объекты `db` и `bcrypt` пока не определены в файле `models`.

Заметим, что объекты db и bcrypt определяются не в этом файле, а в файле `init.py`, так как они передаются в приложение. В `init.py`, как мы помним, уже прописаны необходимые настройки для db и bcrypt. В файле `auth` ChatGPT допустил ошибку, прописав в пятой строке:

```
from app.models import User, db, bcrypt
```

Однако db и bcrypt мы будем брать не из моделей, а из приложения, поэтому конечный фрагмент этого кода имеет строки:

```
from app.models import User
```

```
from app import db, bcrypt
```

Теперь рассмотрим код для файла `models`. Здесь указано верно: база данных импортируется из **app**. Здесь также определяется класс `User`, который наследуется от `db.Model`. В этом классе указаны все поля, которые должны присутствовать в таблице базы данных: `id`, `username`, `password`, `vk_api_id` и `vk_group_id`.

ChatGPT предложил ограничение на длину поля `username` 15. Изменим это значение до 100 символов на случай, если пользовательский никнейм будет длинный. Параметр `unique` указывает, что значение поля `username` должно быть уникальным, то есть не должно быть двух одинаковых записей. Параметр `nullable=False` означает, что поле не может быть пустым. Зададим максимальное значение длины пароля: 80.

Для поля `vk_api_id` установим максимальное значение символов 150 или 200. Оно может оставаться пустым (`nullable=True`). Для поля `vk_group_id` достаточно максимального значения до 20 символов. Также в классе `User` определен метод `repr`, который возвращает базовые значения для таблицы.

Во Flask есть отдельное расширение (Login), которое позволяет прописывать авторизацию ещё проще, но ChatGPT не предложил этот модуль, поэтому мы обойдёмся без него.

Вернёмся к файлу `auth`, в котором была ошибка с `db` и `bcrypt`, и перенесём их на новую строку. Как мы уже говорили, верным здесь будет такой вариант:

```
from app import db, bcrypt  
from app.models import User
```

Продолжаем проверять файл `auth.py` на наличие ошибок. Здесь добавляются Blueprint для авторизации с переменной `auth_bp`, которая передаётся в `init`.

`RegisterForm` — это форма регистрации, в которой запрашивается имя пользователя. Поле `username` обязательно для заполнения благодаря `InputRequired`, и минимальная длина введенного имени должна быть 4 символа. Максимальное значение символов для этого поля установим на 100.

Поле `password` представляет собой `PasswordField`, где вводимый пароль будет отображаться в виде точек. Минимальная длина пароля установлена на 8 символов, и мы оставим это значение. `SubmitField` — это по сути кнопка подтверждения регистрации. В `LoginForm` указаны те же параметры, что и чуть выше, но уже не для регистрации, а для авторизации. Здесь также выставяем максимальные значения для длины имени пользователя 100, для пароля 80.

Мы заметили, что в коде файла `auth` отсутствует строка для подтверждения пароля. Если пользователь опечатается при создании пароля, то в следующий раз он не сможет войти в аккаунт. Можно вписать в код эту строку, однако в этом случае придётся переписывать значительную часть кода. Поэтому обратимся с этой задачей к ChatGPT.

Пишем в ChatGPT запрос: "Нужно подтверждать пароль повторным вводом при регистрации".

ChatGPT создаёт новый код для файла `auth.py`. Удаляем всю информацию из файла `auth` и копируем верный код и снова прописываем числовые значения для длины имени пользователя и пароля.

В строке `confirm` используется `EqualTo`, чтобы убедиться, что введенный пароль совпадает с полем `password`, заданным ранее. Если пароли не совпадают, будет выведено сообщение об ошибке.

Ниже в файле `auth.py` прописан маршрут (route): сначала открывается `RegisterForm`. `hashed_password` представляет собой зашифрованный пароль, который шифруется с помощью `bcrypt`. Параметр `user` берется из данных о его имени (`username.data`) и зашифрованного пароля. Эти данные сразу передаются в модель (`model`).

`db.session.add(user)` отправляет данные в базу данных, а `commit` подтверждает их сохранение. `flash` отвечает за вывод всплывающего окна с сообщением вроде "Регистрация прошла успешно". Если валидация не пройдет, пользователя снова перенаправят на страницу регистрации (`register.html`). Пока этого шаблона не существует, мы вернемся к нему позже.

При авторизации также проводится валидация: пользователь фильтруется через `User.query.filter_by`. Если пользователь существует в базе данных и пароль совпадает с хранящимися данными, создается сессия с прикрепленным `user_id`. Инструмент `session` позволяет работать с базой данных и поддерживать сессию в браузере, чтобы пользователя не выбрасывало из аккаунта при переходе в другое окно. `user_id` берется из базы данных.

В нашем случае данные не сохраняются в cookies, так как эта настройка требует более сложной и долгой работы. Здесь будет достаточно сохранять вход в рамках одной сессии. При закрытии браузера произойдёт окончание сессии и выход из аккаунта.

Если вход в аккаунт прошел успешно, пользователя перенаправляют на dashboard. В противном случае выводится сообщение "Invalid credentials" и пользователь перенаправляется на страницу логина.

Маршрут `@auth_bp.route('/logout')` через `session.pop` удаляет пользователя из сессии. Всплывающее окно сообщает о том, что выход произведен успешно, и происходит перенаправление на страницу входа в аккаунт.

Копируем из ChatGPT код для файла `smm.py`. Детали кода здесь не так важны, потому что мы ещё будем переделывать этот код.

В этом фрагменте кода используются множество модулей:

- Blueprint позволяет организовать маршруты.
- `render_template` используется для отрисовки веб-страниц.
- `request` позволяет обрабатывать запросы форм.
- `flash` используется для отображения всплывающих сообщений.
- `session` хранит данные о сессии пользователя.
- `redirect` перенаправляет пользователя на другие страницы.
- `url_for` генерирует URL для переадресации.

Через `smm_bp` задаются маршруты для Blueprint `smm`. Маршрут `dashboard` проверяет, аутентифицирован ли пользователь. Если пользователь не аутентифицирован (проверяется, есть ли `'user_id'` в `session`), его перенаправляют на страницу логина с помощью `redirect`. Если пользователь аутентифицирован, открывается страница `dashboard`.

Маршрут `settings` работает по аналогичному принципу:

- Если пользователь не аутентифицирован, его перенаправляют на страницу входа.
- Если пользователь аутентифицирован, извлекается его ID из сессии и проверяются данные.

Когда форма отправляется методом POST, обновляются поля `vk_api_id` и `vk_group_id` пользователя. Метод POST используется, когда мы нажимаем кнопку для внесения изменений. После этого вызывается `db.session.commit`, чтобы сохранить изменения в базе данных. Мы изменяем атрибуты пользователя (`user.vk_api_id` и `user.vk_group_id`) на новые значения и сохраняем их.

Для тех, кто знаком с SQL, очевидно, что предложенная в этом уроке конструкция намного проще и эффективнее, чем написание SQL-запросов вручную. Мы просто обновляем нужные параметры и сохраняем изменения.

Сообщение `flash` уведомляет, если сохранение прошло успешно. Затем открывается страница настроек `settings`, где отображаются текущие значения `vk_api_id` и `vk_group_id`.

Иногда могут возникнуть логические ошибки, и код может не работать с первого раза, особенно если генерируется большой блок кода. Обычно нужно просить код небольшими частями и проверять его на работоспособность. Мы будем дорабатывать этот код по мере необходимости, чтобы он работал корректно.

Переходим к коду HTML, копируем его из ChatGPT.

В PyCharm создаём в папке `Templates` новый HTML-файл и даём ему название `register`.

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Register</title>
</head>
<body>
    <h1>Register</h1>
    <form method="POST">
        {{ form.hidden_tag() }}
        <p>
            {{ form.username.label }}

            {{ form.username }}
        </p>
        <p>
            {{ form.password.label }}

            {{ form.password }}
        </p>
        <p>{{ form.submit }}</p>
    </form>
    <p>
        Already have an account?
        <a href="{{ url_for('auth.login') }}">Login here</a>.
    </p>
</body>
</html>

```

Файл `register.html` предназначен для создания страницы регистрации пользователя. В этом HTML-коде используется метод POST для отправки формы на сервер. В заголовке (`<h1>`) указано название страницы — "Register". Форма создаётся с использованием метода POST, что позволяет отправлять данные на сервер.

`{{ form.hidden_tag() }}` создаёт скрытый тег, который защищает форму от CSRF-атак. Поле имени пользователя:

`{{ form.username.label }}` и `{{ form.username }}` отображают метку и поле для ввода имени пользователя.

{ form.password.label }} и {{ form.password }} отображают метку и поле для ввода пароля.

{ form.submit }} отображает кнопку для отправки формы. Внизу формы добавлена ссылка на страницу входа в аккаунт для пользователей, которые уже зарегистрированы.

Таким же образом скопируем файл login.html. В PyCharm создадим в папке templates файл HTML с соответствующим названием.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Login</title>
</head>
<body>
    <h1>Login</h1>
    <form method="POST">
        {{ form.hidden_tag() }}
        <p>
            {{ form.username.label }}

            {{ form.username }}
        </p>
        <p>
            {{ form.password.label }}

            {{ form.password }}
        </p>
        <p>{{ form.submit }}</p>
    </form>
    <p>
        Don't have an account?
        <a href="{{ url_for('auth.register') }}">Register here</a>.
    </p>
</body>
</html>
```

Таким же образом копируем из ChatGPT код для HTML-файла dashboard. Создаём в PyCharm в папке templates файл с соответствующим названием.

Когда будешь называть файлы HTML, внимательно проверяй, нет ли в этих названиях опечаток, так как эти файлы определяют маршруты по своим названиям. Если файл будет иметь другое имя, маршрут не сможет его обнаружить и выдаст ошибку.

Скопированный код вставляем в созданный файл.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Dashboard</title>
</head>
<body>
  <h1>Dashboard</h1>
  <p>Welcome to your dashboard!</p>
  <a href="{{ url_for('smm.settings') }}">Settings</a>
  <a href="{{ url_for('auth.logout') }}">Logout</a>
</body>
</html>
```

Таким же образом создаём файл Settings.html, копируем код из ChatGPT и вставляем:


```

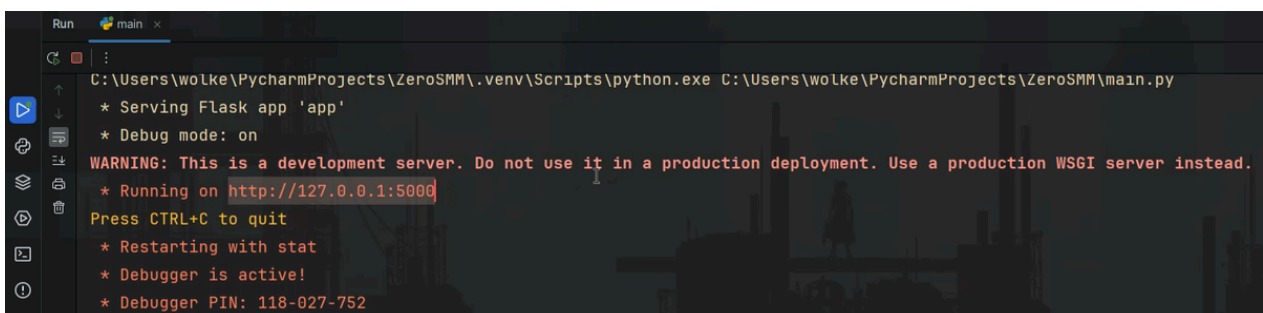
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Settings</title>
</head>
<body>
    <h1>Settings</h1>
    <form method="POST">
        <p>
            <label for="vk_api_id">VK API ID:</label>

            <input type="text" name="vk_api_id" id="vk_api_id" value="{{
</p>
        <p>
            <label for="vk_group_id">VK Group ID:</label>

            <input type="text" name="vk_group_id" id="vk_group_id" value=
</p>
        <p><input type="submit" value="Save"></p>
    </form>
</body>
</html>

```

Запускаем код через main.py (**Run main**). Когда мы запускаем код, здесь отображается адрес, демонстрируется, что debugger активен, выдаётся debugger PIN. Теперь можно в браузере зайти на адрес локал хост, добавив к этому адресу путь: /auth/register



```

Run main
C:\Users\wolke\PycharmProjects\ZeroSMM\.venv\Scripts\python.exe C:\Users\wolke\PycharmProjects\ZeroSMM\main.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 118-027-752

```

При этом, если мы попробуем прописать в адресе `/auth/dashboard`, нас перенаправит на `auth/login`, потому что мы здесь ещё не зарегистрированы.

Теперь в форме можно зарегистрироваться: ввести имя пользователя и пароль. Заметим, если ввести в поле подтверждения пароля значение, не совпадающие с первоначальным паролем, зарегистрироваться не получится, как мы и прописывали. `zero_test`
`qwe123qwe123`.

Теперь, если выйти и снова прописать в адресе `/auth/dashboard`, мы попадём на эту страницу, так как сессия сохранена.

Заменяем `VK_API_ID` и в `VK Group ID` на те, которые содержатся в нашем проекте в файле `config.py`. Видим, что ключ `VK-API` содержит 220 знаков — это выше максимального значения. Возвращаемся в файл `models` и в строке `vk_api_id` увеличим максимально допустимое значение до 250.

Снова запускаем сервер через `main.py`. Переходим в браузер, обновляем страницу и вставляем в поле `VK API ID` скопированный ключ.

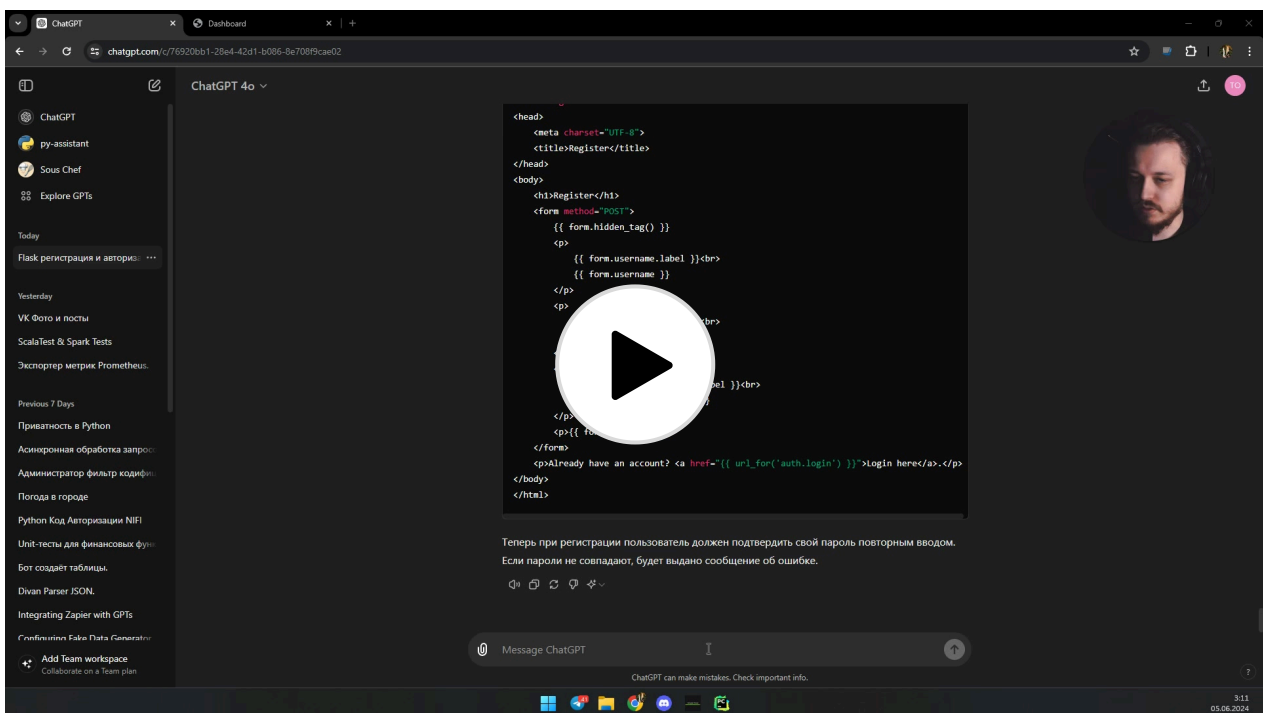
Теперь скопируем из файла `config.py` `vk_group_id` и вставим это значение в соответствующее окно на сайте. Нажимаем **Save**.

Обращаем внимание на то, что в нашем случае мы не задали требование введения адреса электронной почты, но в целом добавить на сайт этот запрос не составляет труда: во-первых, можно попросить ChatGPT добавить почту в `models`; также можно вручную в файле `models.py` ввести e-mail (при вводе начала этой строки модуль `Kodeium` выдаст соответствующую подсказку). Затем в файле `auth.py` нужно будет прописать поле `EmailField`, чтобы правильно валидировать этот параметр. Также можно прописать в коде

требование подтверждения электронной почты. Это всё также прописывается в `models`, но сегодня мы не будем посвящать этому время.

Итак, мы создали функционирующее приложение с возможностью регистрации, входа в аккаунт и сохранения сессии. В следующей части урока мы создадим несколько HTML-страниц, отвечающих за генерацию постов и автопостинг.

4. Подключение функциональной части и обработка ошибок



Приступаем к подключению функциональной части. Если при запуске программы возникает ошибка, она сразу отобразится внизу. В этом случае сайт не откроется. Нужно скопировать код ошибки (лучше всего скопировать целиком весь трейсбэк) и отправить его в ChatGPT для анализа и исправления. Иногда может потребоваться повторить этот процесс несколько раз. При этом **Warning** — это не ошибки, а предупреждения.

Если ошибки будут содержаться в логике открытия страниц, то это ошибка будет видна не только в трейсбэке, но и в браузере: при попытке открытия страницы отобразится **Error** с причиной возникновения неполадок — можно сделать скриншот этой страницы и также отправить в ChatGPT с просьбой разъяснить суть проблемы.

Теперь переходим к функционалу генераторов. Мы будем подключать генератор текста и другие необходимые модули. Снова обратимся к ChatGPT.

Сначала копируем последовательно информацию из нашего проекта, а именно, из файлов `image_gen.py`, `text_gen.py` и `vk_publisher.py`. Перед каждым копированием даём пояснение для ChatGPT: "Вот `text_gen.py` из проекта", "Вот функционал `image_gen.py` из проекта", "Вот функционал `vk_publisher.py` из проекта".

Перед этим текстом пишем основной запрос в ChatGPT: "Теперь нужно сделать ещё одну страничку и кнопку в dashboard. Это страница "post generator". В нём базово должно быть два поля: для задания тона и темы поста. Дополнительно нужно два чек-бокса (галочки) для генерации картинки и для автопостинга в VK. Если поставить галочку на генерации картинки, то на выход нам даст не только текст сгенерированного поста, но и картинку. Если стоит галочка на autopубликации, то должен быть выполнен автопостинг в VK, куда передается текст поста и картинка (если она была сгенерирована). Сохранять пост, тон и картинку в БД не нужно". Также добавим: "Ключ OpenAI лежит в `config.py` в корне проекта".

ChatGPT создал новые маршруты для генерации постов. Маршрут `post_generator` проверяет наличие `user_id` в сессии. Затем объект `post_gen` использует ключ API, заданный тон и тему для генерации поста. Переменная `image_url` изначально равна `None`. Если установлена галочка `generate_image`, подключается `ImageGenerator`, который генерирует изображение на основе `image_prompt` и возвращает ссылку на это изображение, присваивая её `image_url`. Если установлена галочка `auto_post`, подключается `VKPublisher`,

который публикует пост и изображение, после чего выводит всплывающее окно (flash) с уведомлением об успешной публикации. В конце с помощью `render_template` заново отрисовывается страница, передавая `post_content` и `image_url`, если они имеются.

Также ChatGPT прописал `dashboard.html` с добавленной новой ссылкой — `smm.post_generator`. Копируем этот код и вставляем его в файл `dashboard.html` в проекте.

Также ChatGPT предлагает создать новый файл `post_generator.html`. Создадим его в проекте в папке `templates`. Копируем код из ChatGPT и вставляем в файл.

Запускаем код через `main.py`. Заходим в браузер и, если сессия ещё не потеряна, обновляем страницу. Видим, что добавилась новая вкладка: **Post Generator**. Нажимаем на эту вкладку и видим здесь то, что прописывали: поле для ввода **Tone** и поле для ввода **Topic**, а также два чекбокса и кнопка **Generate**.

Проверим работоспособность страницы. В поле **Tone** напишем, например: "Задорный", в поле **Topic** — "Новая коллекция точилок для ножей Zero knives". Нажимаем **Generate**. Через короткое время получаем чуть ниже сгенерированный пост.

Теперь сделаем то же самое, но добавим галочку в поле **Generate Image**. Под текстом через некоторое время появляется изображение, сгенерированное ИИ.

Не меняя тона и темы, добавим галочку **Auto Post to VK**. Нажимаем **Generate**. На странице через некоторое время готовы и новый пост, и новая картинка. Заходим на страницу ВКонтakte Zero Knifes — пост уже опубликован. Всё работает.

Переходим к созданию кода для извлечения статистики. Вновь обратимся к ChatGPT с запросом: "Теперь надо сделать ещё одну страницу и кнопку в `dashboard` — `VK Stats`". Хочу, чтобы он

вытаскивал количество подписчиков с помощью нашего `vk_stats`. На странице должна быть табличка с параметрами сообщества ВК, которые можно вытаскивать из статистики. Выдавай туда пока только подписчиков, но можно сделать заглушки с незадаанными значениями, которые могут появиться впоследствии. Вот код нашего `vk_stats.py`, который лежит в папке `social_stats` в корне проекта”, затем копируем код из названного файла. Однако копируем мы этот код не полностью: удаляем строки кода, в которых передаётся конфигурация (последние строки кода `“if name == main”`), а также `“import time”` и `“import config as conf”` вначале. Вставляем скопированный код и добавляем его к запросу. Отправляем запрос.

ChatGPT создал новый файл `smm.py`, но мы не будем его полностью копировать, а возьмём только новую информацию. Копируем строку: `from social_stats.vk_stats import VKStats` и вставляем её в `smm.py` сразу после строки `“import VKPublisher”`.

Также после `post_generator` вставляем новый маршрут для `VK Publisher`, копируя соответствующий фрагмент кода.

Так же, как и в остальных частях, здесь код передаёт сессию пользователя, черпает сведения из базы данных пользователей, извлекает статистику по `vk_stats`. `stats` здесь отображено в виде словарика, где параметры для лайков, комментариев и репостов не заданы, так как мы не давали такой задачи.

В файл `dashboard` тоже нужно внести изменения: копируем соответствующую строку, касающуюся `vk_stats` и вставляем в файл `dashboard.html` в проекте.

Создадим ещё один файл в проекте: `vk_stats.html`. Копируем код для этого файла из ChatGPT и вставляем его в этот файл.

Запускаем код через `main.py` и, зайдя в браузер, обновим страницу. В дашборде появилась новая вкладка `VK_Stats`. Нажимаем на неё и видим табличку, в которой указано количество подписчиков.

На странице VK Stats также есть полезная вкладка **Back to Dashboard**. Такой кнопки нет на других страницах. Заходим в файл `vk_stats.html` и копируем эту строчку кода. Добавим эту строчку в код для `post_generator.html` и `settings.html`.

Снова запускаем код через `main.py`. Обновляем страницу в браузере и видим, что эта вкладка добавилась везде, где нам было нужно.

Также можно прописать отдельный маршрут для главной страницы. Для этого копируем код из `init.py` и обращаемся к ChatGPT с запросом: "Хочу, чтобы тут был маршрут главной страницы, который будет делать редирект на dashboard". Для этого копируем и вставляем в `init.py` следующий код:

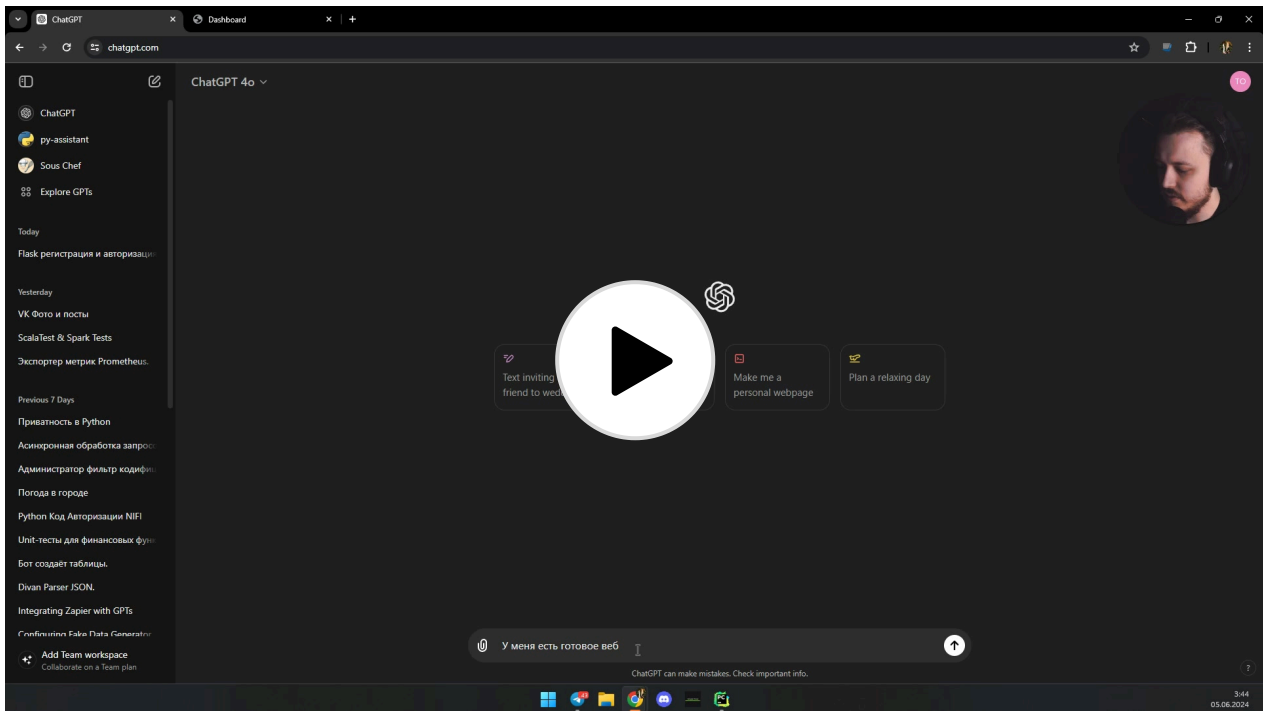
```
@app.route('/')  
def index(): return  
    redirect(url_for('smm.dashboard'))
```

Теперь, перезапустив `main.py`, мы увидим, как это работает. Обновляем страницу, и нас сразу направляет на **Dashboard**. При попытке открыть **Dashboard** без аккаунта мы попадём на страницу ввода логина и пароль.

Теперь в приложении есть настройки, где мы можем ввести VK API и VK Group ID. Здесь также есть возможность генерация постов. В принципе, этим приложением можно пользоваться, однако сейчас наше приложение запускается локально: оно нигде не опубликовано и к нему нет доступа.

В следующей части урока перейдём к созданию стиля для приложения. Для этого нам опять понадобится ChatGPT и готовые библиотеки для генерации стилей к веб-сайтам — например, Bootstrap.

5. Создание стилизованных HTML-шаблонов с Bootstrap



В новом чате напишем ChatGPT: “У меня есть готовое веб-приложение на Flask с уже готовыми HTML-шаблонами, но они все без стилей. Хочу узнать, какие инструменты есть, чтобы сделать их стильными быстро и легко без знания и написания CSS”.

ChatGPT предложил несколько вариантов:

- **Bootstrap.** Это фреймворк для CSS, который позволяет быстро создавать стилизованные веб-страницы. Он упрощает процесс задания стилей, используя классы. Пример подключения Bootstrap включает в себя HTML-код с комментариями и контейнерами.
- **Materialize CSS.** Он менее удобен в использовании по сравнению с Bootstrap.
- **Bulma.** Это ещё один CSS-фреймворк, который часто называют библиотекой. Он подключается так же, как и Bootstrap.

Надеясь на то, что ChatGPT помнит контекст диалога, вернёмся в предыдущий чат и напишем новый запрос: “А теперь надо сделать стили для всех HTML-файлов. Я не хочу писать CSS код, поэтому давай использовать Bootstrap. Сделай всё красиво. Наши шаблоны: login.html, register.html, dashboard.html, settings.html, post_generator.html, vk_stats.html”. Отправляем запрос.

ChatGPT создал базовый шаблон, от которого будут наследовать все остальные. Это пример использования Jinja, шаблонизатора, который позволяет передавать различные значения из Flask в HTML-страницы. Jinja также позволяет создать единый HTML-шаблон для всей структуры сайта, а на остальных страницах прописывать только отличающиеся элементы.

Создаём в нашем проекте в папке templates новый файл: base.html, копируем предложенное ChatGPT и вставляем в этот файл.

За счёт строки `extends "base.html"` он добавляет всё, что отсутствовало в файле. В `block content` он расположит страницы и поместит туда содержимое.

Bootstrap работает с контейнерами, поэтому в коде мы видим использование контейнеров. Это означает, что разметка сайта будет ровной.

Последовательно меняем информацию в html-файлах в проекте на ту, которую предложил ChatGPT.

Когда код для всех html-файлов скопирован и вставлен в соответствующие файлы в проекте, можно запускать код через `main.py`. Заходим в браузер и обновляем страницу. Войдём в аккаунт. Стили применены, но потеряна кнопка **Back to Dashboard**. Но так как теперь на дашборд можно выйти, нажав на главную страницу, можно просто убрать эту строчку из кода в файле `vk_stats.html`. Перезапускаем код, обновляем страницу. Теперь всё в порядке, приложение выглядит удобным и имеет привлекательный интерфейс.

Попробуем сгенерировать пост и картинку. Снова зададим тон и тему, поставим галочку в поле генерации картинки. Через некоторое время получаем пост и картинку большого размера под ним.

Если нужно ограничить размеры картинки, это тоже прописывается через html. Для этого можно просто сбросить в ChatGPT код из post_generator.html и отправить с кодом запрос о том, что необходимо задать картинки заданного размера. Можно и вручную прописать в файле post_generator.html в строке кода параметры картинки — например, зададим ей значение **300**:

```
  
(<https://www.pythonanywhere.com/>)

*Если по какой-то причине ты не можешь воспользоваться прямой ссылкой, то эту платформу легко можно найти в поиске Google.*

На главной странице сайта нажимаем кнопку **Start running Python online!** На следующей странице нажимаем **Create a Beginner account**.

В форме регистрации указываем имя пользователя, адрес электронной почты и пароль. Ставим галочку, соглашаясь с условиями использования, и нажимаем **Register**.

Здесь можно пройти небольшое знакомство с интерфейсом, но сейчас мы не будем его проходить и нажмём **End Tour**.

Переходим в свою почту, подтверждаем регистрацию. Начинаем создавать приложение. Нажимаем вкладку **Web** на верхней панели справа. Нажимаем **Add a new web app**. Нажимаем **Next**. На следующем шаге выбираем фреймворк — **Flask**. Выбираем версию Python 3.10. В поле **Path** дописываем в конце пути main.py (<http://main.py>). Нажимаем **Next**. Приложение создано, у него уже есть адрес.

Чтобы загрузить файлы, переходим во вкладку **Files** (расположенную на верхней панели сайта). Файлы нужно загружать поочерёдно, создавая отдельные папки. Нажимаем **Upload a file** и загружаем файл из проекта.

Чтобы перед загрузкой файлов с компьютера не искать проект, можно скопировать его путь: переходим в PyCharm, находим вкладку **Copy Path/References — Absolute Path** и в проводнике вставляем этот путь в адресную строку. Подключаем config.py (<http://config.py>).

В main.py (<http://main.py>) на сайте пока вставим наш код, но уберём фрагмент кода "if Name == main". Сохраняем изменения и возвращаемся обратно на страницу загрузок.

Чтобы создать папку, пишем её название в текстовом поле и нажимаем **New directory**. Создадим таким образом папке social\_stats. Сразу загрузим в неё файлы, которые имеют к ней отношение через **Upload a file**.

Возвращаемся обратно в папку приложения и создаём следующую папку: social\_publishers и загружаем в неё нужные файлы.

Папку instance из проекта можно не перекидывать, потому что всё равно на платформе создастся новая база данных. Однако, если мы хотим, чтобы все данные о пользователях заранее были в проекте, всё-таки загрузим эту папку.

Чтобы не ошибиться и не опечататься, давая названия папкам на хостинге, можно, находясь в проекте, нажать **Refactor — Rename**, после чего скопировать название этой папки через **Ctrl-C** и вставить это название в поле **New directory**.

Загружаем последовательно все папки проекта. Каждый раз необходимо дождаться завершения загрузки, и только потом снова нажимать **Upload a file**. К сожалению, здесь отсутствует функция одновременной загрузки нескольких папок или файлов.

Хоть папка static в нашем проекте пуста, её также необходимо создать на платформе. Здесь она тоже будет пустой, но путь к ней в любом случае должен быть.

Когда все папки загружены, мы можем попытаться запустить приложение. Возвращаемся во вкладку **Web** и нажимаем кнопку **Reload**. Переходим на выданный адрес приложения и видим ошибку (Something went wrong). Также здесь есть лог ошибок — можно заглянуть в него, чтобы посмотреть, в чём проблема.

В нашем случае проблема в том, что модуль OpenAI автоматически не установился в модуль. Его нужно установить вручную.

Чтобы установить OpenAI, заходим во вкладку **Consoles**, выбираем версию Python (3.10) и открываем **Bash**-консоль.

В консоли для начала проверим, что у нас уже есть, для этого напишем: **pip list**. Выводится очень много модулей, в том числе и для Flask — это значит, что для Flask ничего дополнительно скачивать не нужно.

Установим модуль OpenAI. Пишем: **pip install openai**. Ждём некоторое время, пока завершится установка.

Снова напишем **pip list**, чтобы проверить, установился ли OpenAI. Он появился в списке.

Возвращаемся во вкладку **Web** и перезапускаем приложение кнопкой **Reload**.

На этой странице можно настроить дополнительные параметры, настроить характеристики, менять файлы логов, а также задать путь к папке Static (но нам это не нужно, так как эта папка не используется). Можно включить HTTPS, чтобы сайт открывался защищённым протоколом.

Снова заходим на адрес, выданный нашему приложению. Всё заработало, сайт открылся. Теперь можно сбрасывать эту ссылку кому угодно и продвигать этот сайт. Страница опубликована и функционирует.

Попробуем зарегистрироваться на странице. Переходим на форму регистрации. заполняем необходимые поля. Аккаунт зарегистрирован.

Заходим на страницу и видим все заданные нами вкладки и формы. Сейчас генератор ничего не сгенерирует, потому что ни VK API, ни VK Group ID здесь ещё не заданы.

**Подведём итоги.** Мы проделали большую работу. Это всё может показаться запутанным, но на самом деле всё это до конца понимать и не нужно, так как по тем или иным вопросам можно обращаться к ChatGPT.

Если тебе интересно создавать подобные веб-приложения, можно найти в сети дополнительную информацию (например, “Мега-учебник Flask”, о котором говорилось в начале урока), и тогда почти всё, что мы здесь делали, тебе будет понятно.

Что касается работы со стилями, возможно, вручную, имея навыки разработки, можно сделать даже лучше, чем в нашем примере сделал фреймворк Bootstrap. Но Bootstrap на самом деле очень удобен и многофункционален. Кроме того, в работе с Bootstrap нам тоже может помочь ChatGPT. Достаточно просто отправить ему нужные HTML-страницы и написать в запросе, что мы хотим применить здесь стили через Bootstrap.

Вместе с тем в любом случае не следует во всём полагаться на ChatGPT, так как порой он выдаёт ошибки, которые в уроке мы исправляли на ходу. Но для этого достаточно поверхностного понимания предмета.

Подобные приложения, конечно, возможно разработать и nocode-инструментами, без Flask и других фреймворков. Но nocode представлен арсеналом ограниченных инструментов, а с помощью программирования можно не ограничивать себя и реализовывать всё, что приходит в голову.

Как мы видим, на создание этого приложения у нас ушло всего несколько часов. При этом мы не копировали заранее заготовленные шаблоны, а вводили все запросы в режиме реального времени. Процесс от идеи до реализации в итоге занял пару уроков и в итоге мы получили абсолютно рабочее приложение, которым можно пользоваться и которое можно рекомендовать другим людям.

Успехов в реализации проектов и в освоении промпт-инжиниринга!

Итоговый код для приложения SMM-эксперт.zip  
(<https://fs24.getcourse.ru/fileservice/file/download/a/256825/sc/294/h/3ed5cb65576b64cbf8b5d4a9176eba05.zip>).

## **Мега-учебник по Flask:**

<https://habr.com/ru/amp/publications/804245/>  
(<https://habr.com/ru/amp/publications/804245/>)

## **Домашнее задание**

Используй итоговый код для приложения SMM-ассистента.

Создай базовое веб-приложение с использованием Flask, реализующее функционал генерации постов и изображений с последующей автопубликацией в ВКонтакте. Приложение должно поддерживать регистрацию и авторизацию пользователей.

Предоставь ссылку на развернутое приложение на PythonAnywhere.

## **На следующем занятии**

Поработаешь с Text-to-Speech моделью и сделаешь помощника для подкастов и озвучки коротких форматов видео.

Если у вас есть вопросы, задавайте их, пожалуйста, в учебном чате, а не в этой анкете, — так вы сможете быстрее получить ответ.

Урок



### Понравился ли урок?

- ☐ Понравился, все было понятно
- ☒ Неплохо, но не все получилось/не все понятно
- ☐ Не понравился

### Обратная связь по уроку (не обязательно)

Отправить

## ЗАДАНИЕ

Используй итоговый код для приложения SMM-ассистента.

Создай базовое веб-приложение с использованием Flask, реализующее функционал генерации постов и изображений с последующей автопубликацией в ВКонтакте. Приложение должно поддерживать регистрацию и авторизацию пользователей.

Предоставь ссылку на развернутое приложение на PythonAnywhere.





Прикрепить файлы

максимальный размер файла - 100мб

Отправить ответ

Сохранить как черновик

20 монет за выполнение этого урока

---

Предыдущий  
урок

(<https://university.zerocoder.ru/pl/teach/control/lesson/view?id=338667526>)

Следующий урок